

ZEN: Empowering Distributed Training with Sparsity-driven Data Synchronization

Zhuang Wang, Zhaozhuo Xu, Jingyi Xi, Yuke Wang, Anshumali Shrivastava, T. S. Eugene Ng

Presented to

Dr. Muhammad Abdullah
Adnan
CSE, BUET

Presented by

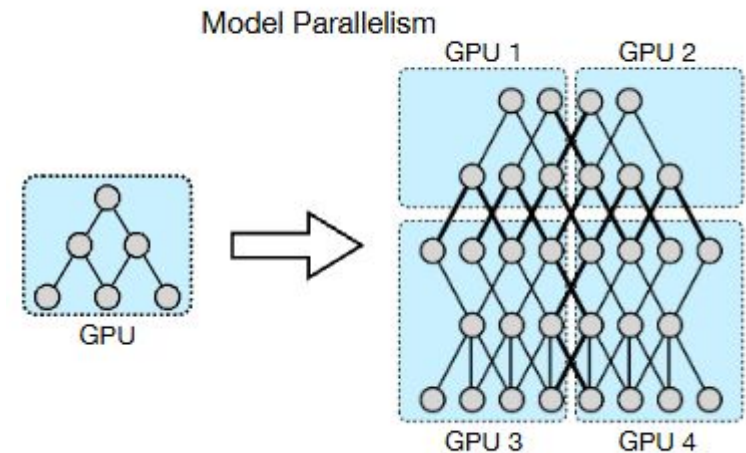
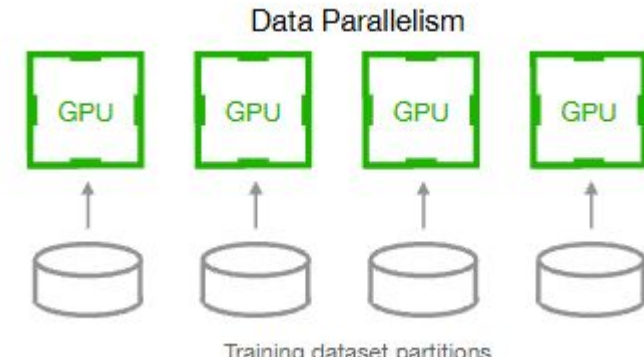
Rahul Drabit
Chowdhury
ID : 01858017317



Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

Trends in Deep Learning

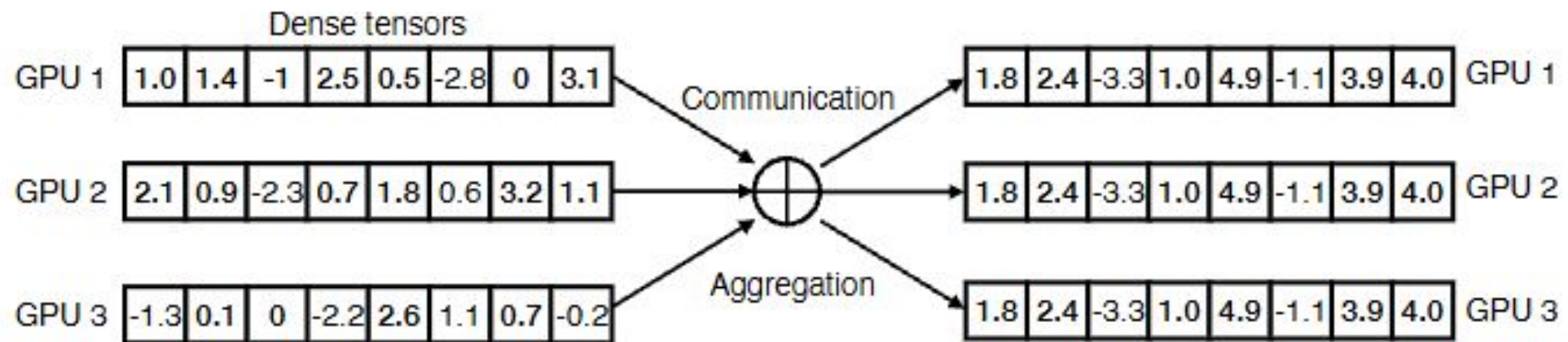
- Data grows exponentially
 - Data Parallelism
- Model size grows exponentially
 - Scale to trillion parameters
 - Model Parallelism: TP, PP, FSDP
- Hybrid parallelism for distributed training
 - Data Parallelism + Model Parallelism



Gradient synchronization

Dense tensor synchronization among GPUs

- Communication and aggregation



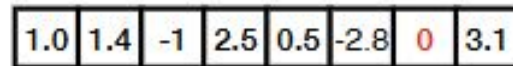
- Synchronization of dense tensors has been well studied
 - BytePS [1] and Ring-AllReduce [2]



Sparsity in gradient tensors

Zero gradients in tensors

- Gradients can be zeros



Low sparsity

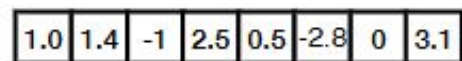


High sparsity

- Gradients can be zeros
- Natural tensor sparsity in popular models

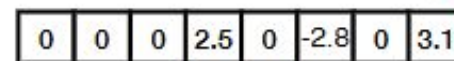
Model	MLP Size	Embedding Size	Embedding Sparsity
LSTM	20M	406M	98.87%
DeepFM	68M	214M	97.20%
NMT	31M	112M	97.53%

- Tensor sparsity from gradient sparsification algorithms [1,2,3]



Low sparsity

Sparsity →



High sparsity

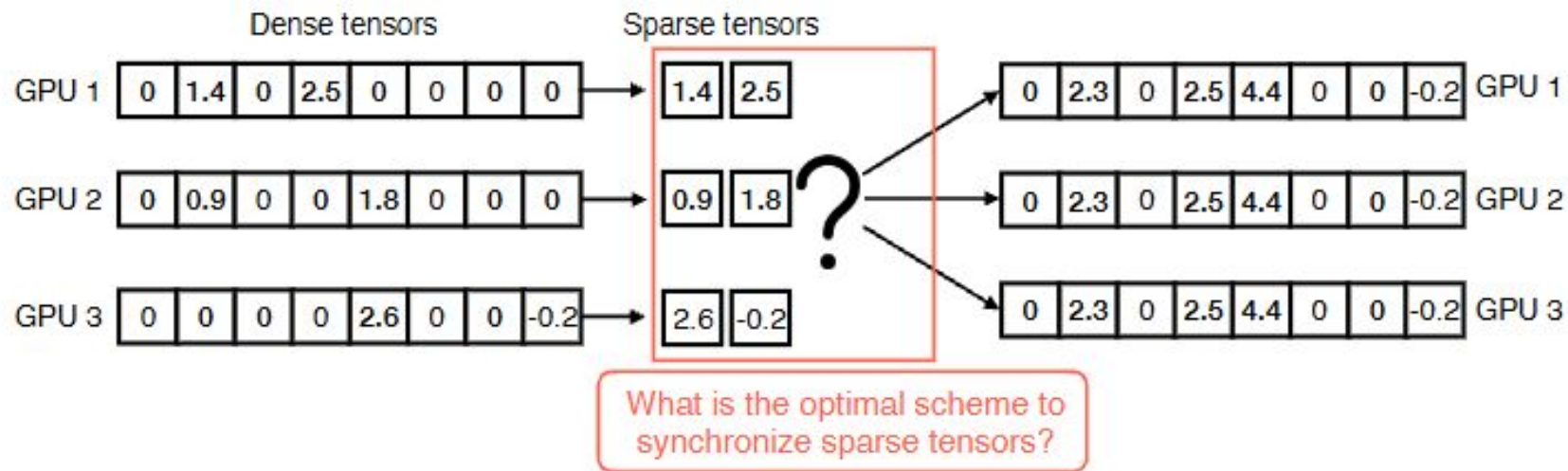
Up to 99% sparsity



Synchronization of sparse tensors

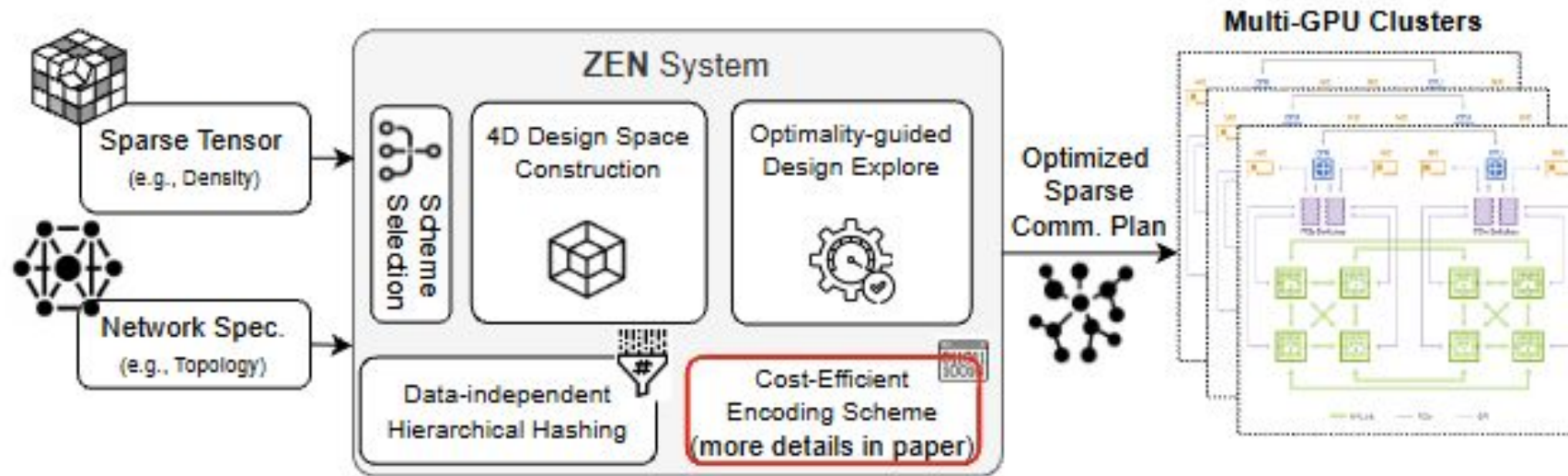
Opportunities and Research Problem

- Only communicate sparse tensors, i.e., non-zero gradients
 - Greatly reduces traffic volume and synchronization time



ZEN

A near-optimal synchronization system



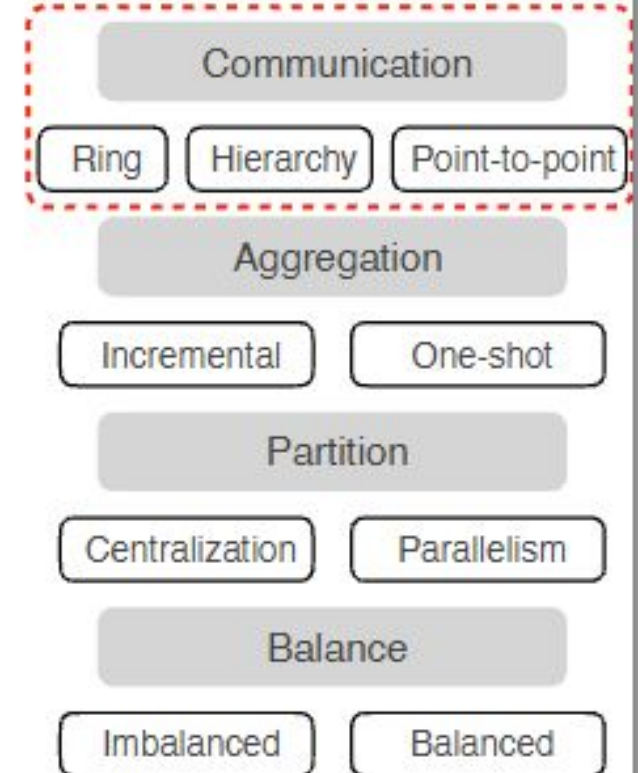
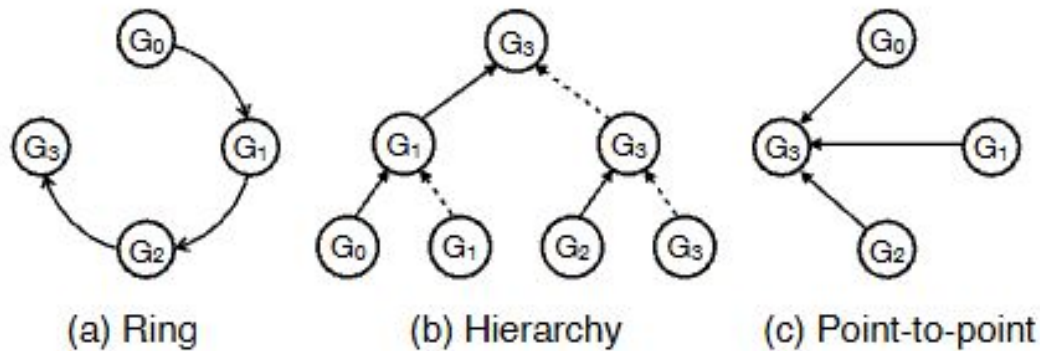
Design space construction

Four elemental dimensions

- Sparse tensor before synchronization



- Communication dimension



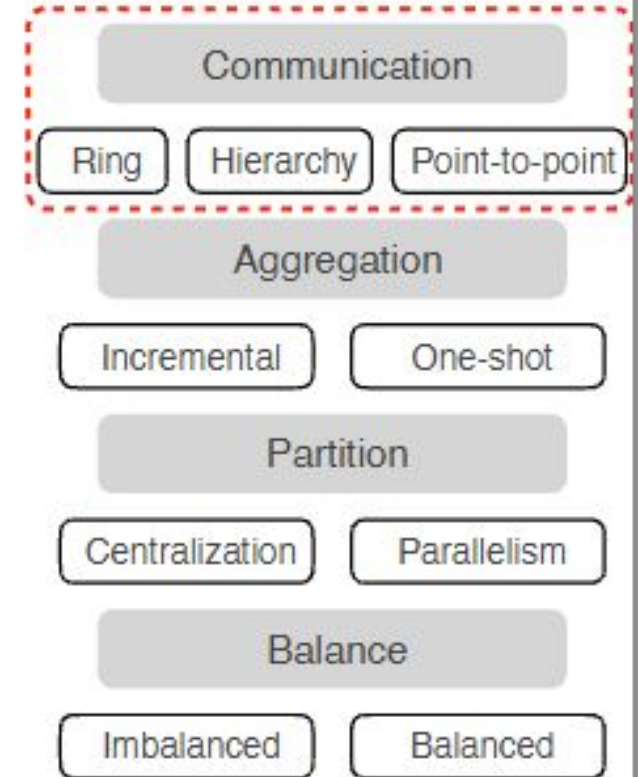
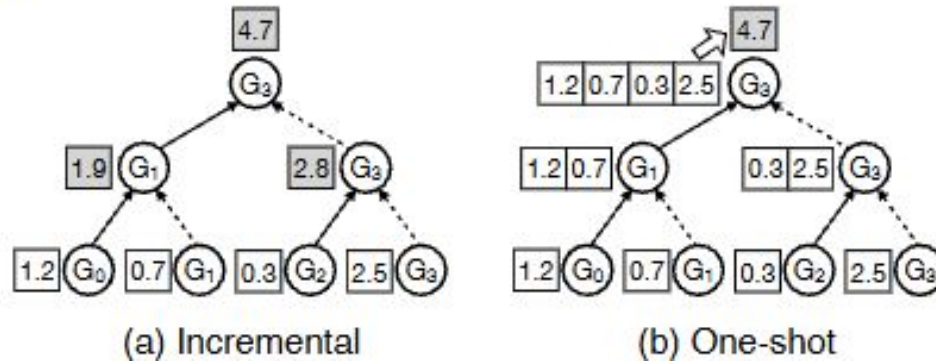
Design space construction

Four elemental dimensions

- Sparse tensor before synchronization



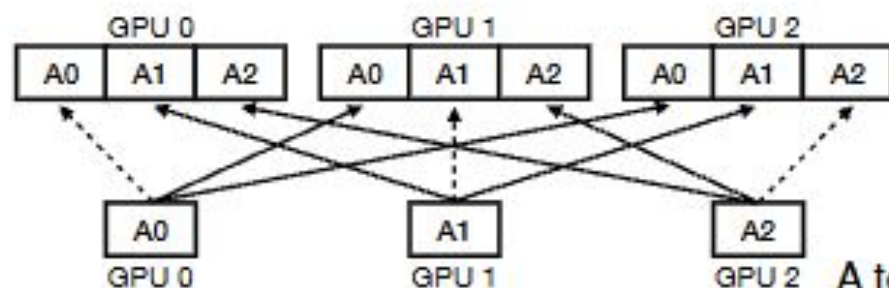
- Aggregation dimension



Design space construction

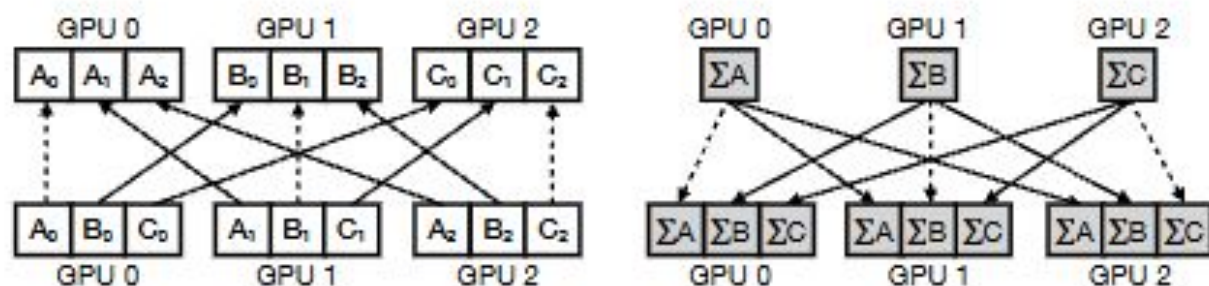
Four elemental dimensions

- Partition dimension



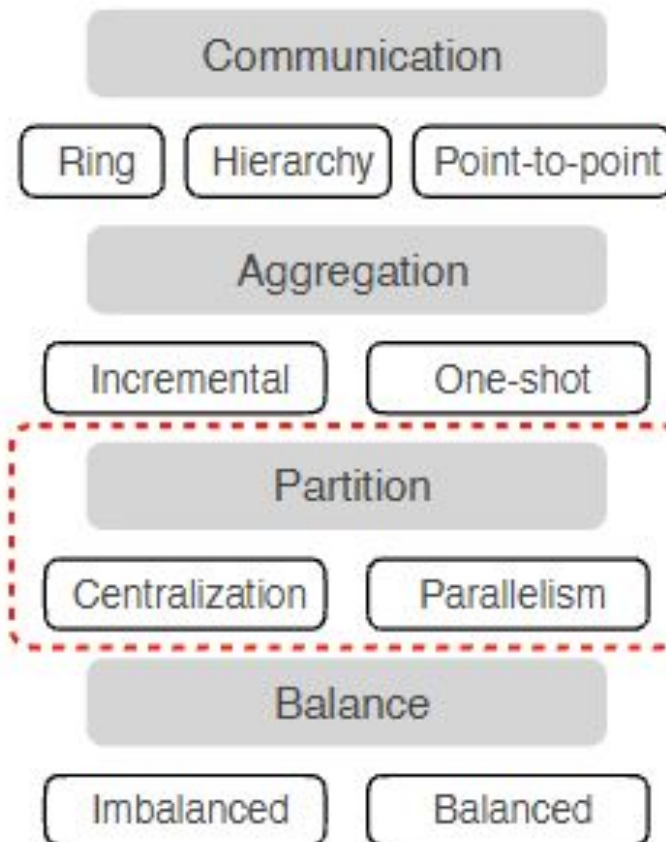
(a) Centralization

A tensor is communicated as a whole



(b) Parallelism

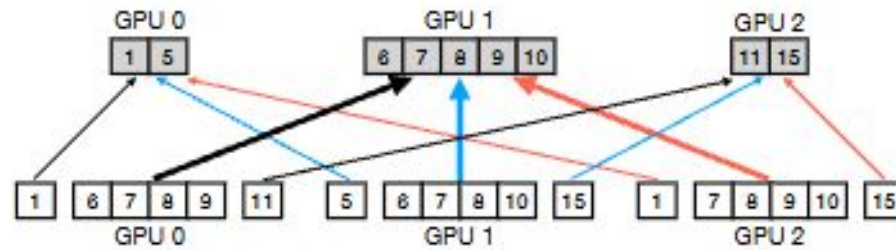
A tensor is partitioned before communication



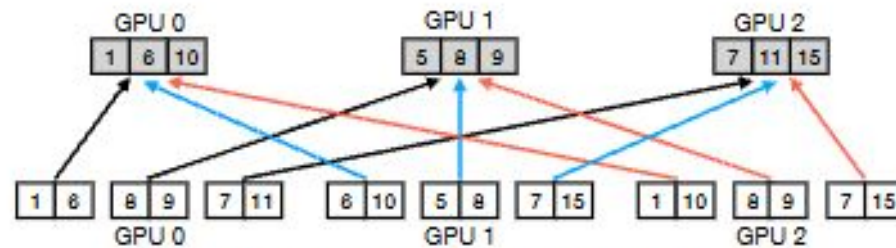
Design space construction

Four elemental dimensions

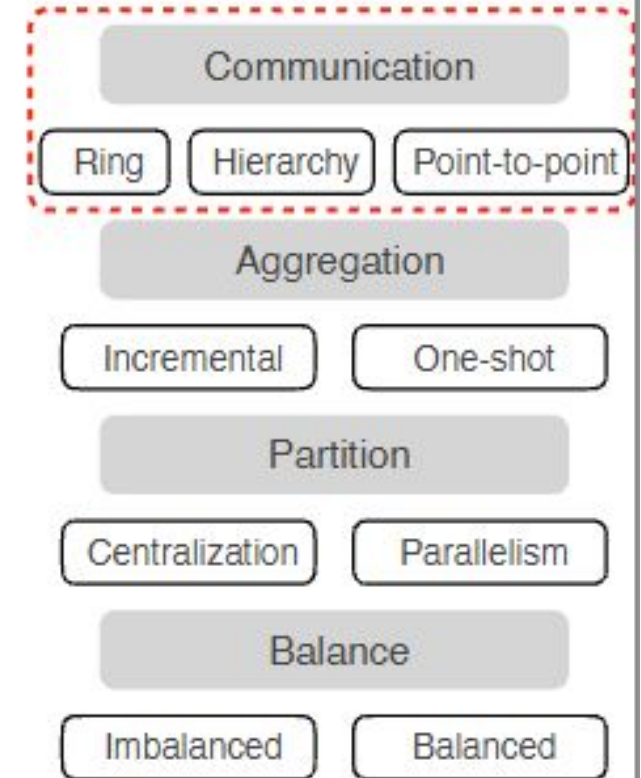
- Balance dimension



(a) Imbalanced communication



(b) Balanced communication



Design space construction

Expressiveness

- Existing schemes can be described by the four dimensions

Schemes	Communication	Aggregation	Partition	Balance
AGsparse [1]	Ring, Hierarchy, Point-to-point	One-shot	Centralization	N/A
SparCML [2]	Hierarchy	Incremental	Centralization	N/A
OmniReduce [3]	Point-to-point	One-shot	Parallelism	Imbalanced

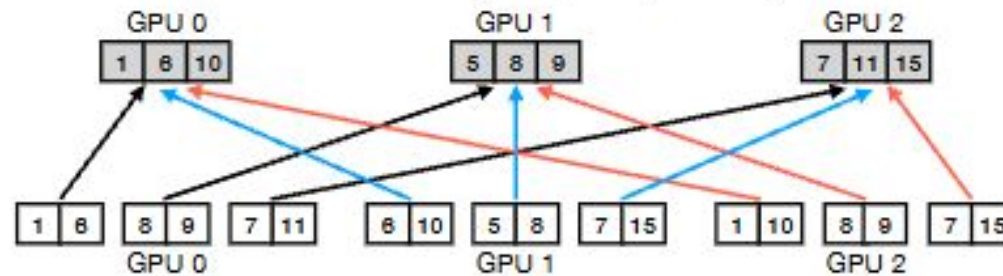
- The four dimensions can describe the whole design space



Find the optimal schemes

Based on the four dimensions

- “Balanced Parallelism”



“Balanced Parallelism”

Communication

Ring

Hierarchy

Point-to-point

Aggregation

Incremental

One-shot

Partition

Centralization

Parallelism

Balance

Imbalanced

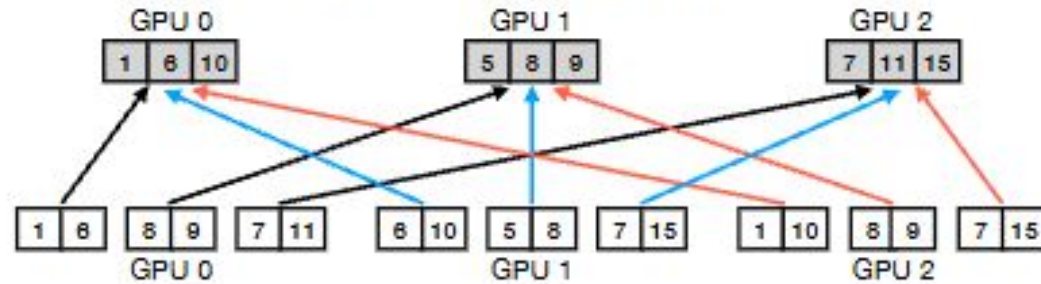
Balanced



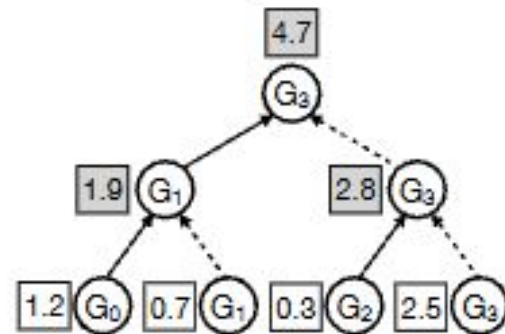
Find the optimal schemes

Based on the four dimensions

- “Balanced Parallelism”



- “Hierarchical Centralization” (SparCML [1])



Balanced Parallelism as the practical optimal scheme

No existing realization of Balanced Parallelism

“Hierarchical Centralization”

Communication

Ring

Hierarchy

Point-to-point

Aggregation

Incremental

One-shot

Partition

Centralization

Parallelism

Balance

Imbalanced

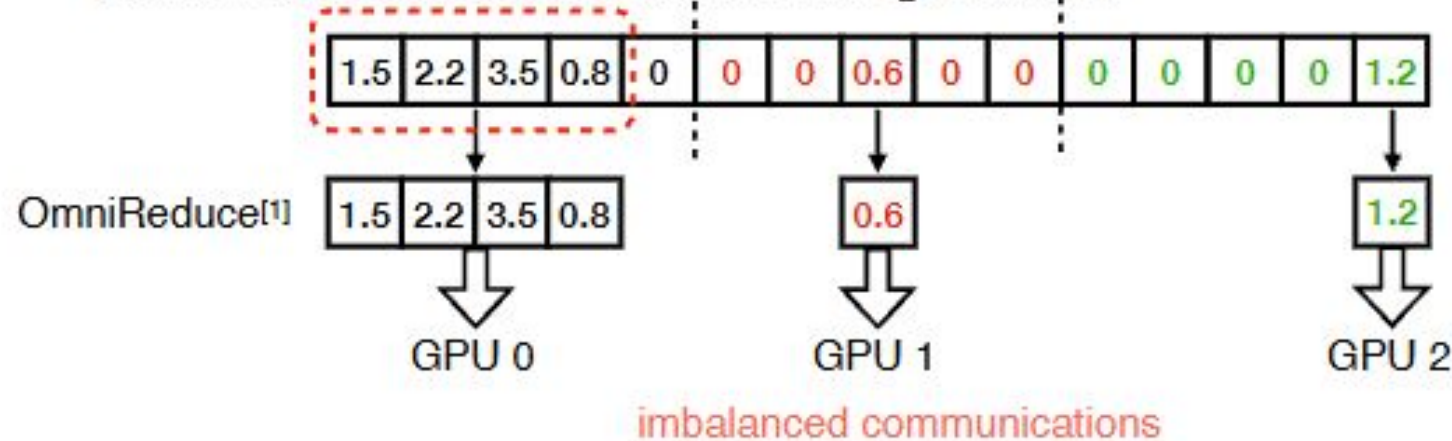
Balanced



How to realize “Balanced Parallelism”?

Imbalanced communication

- Skewed distribution of non-zero gradients



Communication

Point-to-point

Aggregation

One-shot

Partition

Parallelism

Balance

Balanced



Communication time analysis

• **Balanced Parallelism.** Communication in this scheme involves two steps. The traffic volume each GPU receives is $\frac{2(n-1)}{n}d_G M$ in the first step and $\frac{2(n-1)}{n}\gamma_G^n d_G M$ in the second step. The total traffic volume that each GPU receives is $\frac{2(n-1)}{n}(\gamma_G^n + 1)d_G M$, resulting in a communication time of:

$$T_{bp} = \frac{n-1}{n}(\gamma_G^n + 1)2Md_G/b. \quad (1)$$

• **Hierarchical Centralization.** Communication in this scheme is performed in $\log n$ stages. In the i_{th} stage, the traffic volume that each GPU receives is $2\gamma_G^{2^{i-1}}d_G M$. The total traffic volume that each GPU receives is $2\sum_{i=1}^{\log n}\gamma_G^{2^{i-1}}d_G M$. The corresponding communication time is:

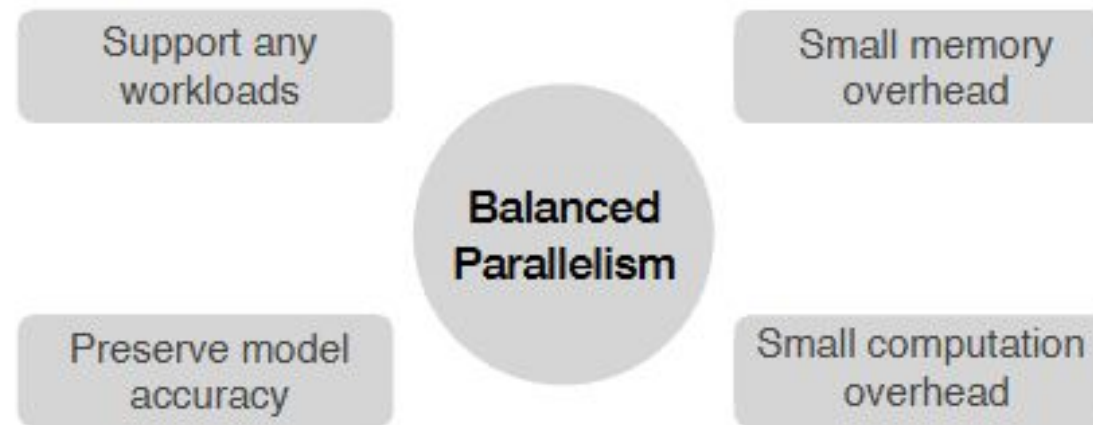
$$T_{sm} = \sum_{i=1}^{\log n}\gamma_G^{2^{i-1}}2Md_G/b. \quad (2)$$



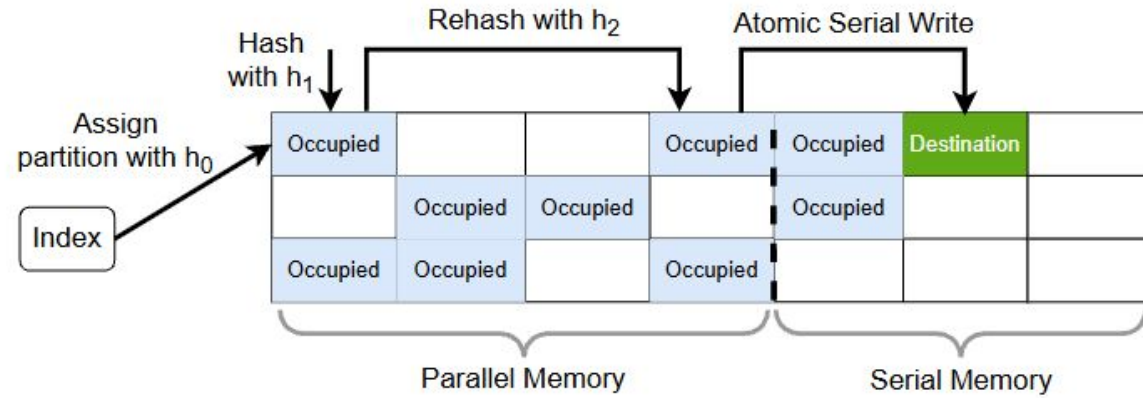
How to realize “Balanced Parallelism”?

Challenge and requirements

- Challenge: How to achieve balanced communications?
- Requirements



Algorithm 1: Hierarchical Hashing Algorithm



Algorithm 1: Hierarchical Hashing Algorithm

Input: G is a dense tensor and $I \subset \mathbb{N}_+$ is the indices of its non-zero gradients. n is the number of partitions. Each partition has a memory size $r_1 + r_2$, where r_1 and r_2 are the memory sizes for parallel and serial operations, respectively. $h_0: \mathbb{N}_+ \rightarrow [n]$ is a universal hash function. $H = \{h_1, \dots, h_k\}$ is a set of universal hash functions where $h_i: \mathbb{N}_+ \rightarrow [r]$.

Output: The partitioned sparse tensors.

```

1 Function hierarchical_hash( $I, G, h_0, H$ ):
2   Allocate memory  $x \leftarrow \mathbf{0}^{n \times (r_1 + r_2)}$ 
3   Allocate atomic counters  $c \leftarrow \mathbf{r}_1^n$ 
4   foreach  $idx \in I$  in parallel do
5      $p \leftarrow h_0(idx)$ 
6     for  $i \leftarrow 1$  to  $k + 1$  do
7        $q \leftarrow h_i(idx)$ 
8       if  $i = k + 1$  then
9          $q \leftarrow \text{atomicAdd}(c[p], 1)$ 
10         $x[p][q] \leftarrow idx$ 
11      end
12      if  $x[p][q] == 0$  then
13         $x[p][q] \leftarrow idx$ 
14        break
15      end
16    end
17  end
18   $output = []$ 
19  for  $i \leftarrow 0$  to  $n - 1$  do
20     $indices = \text{nonzero}(x[i])$ 
21     $values = G[indices]$ 
22     $output.append((indices, values))$ 
23  end
24  return  $output$ 

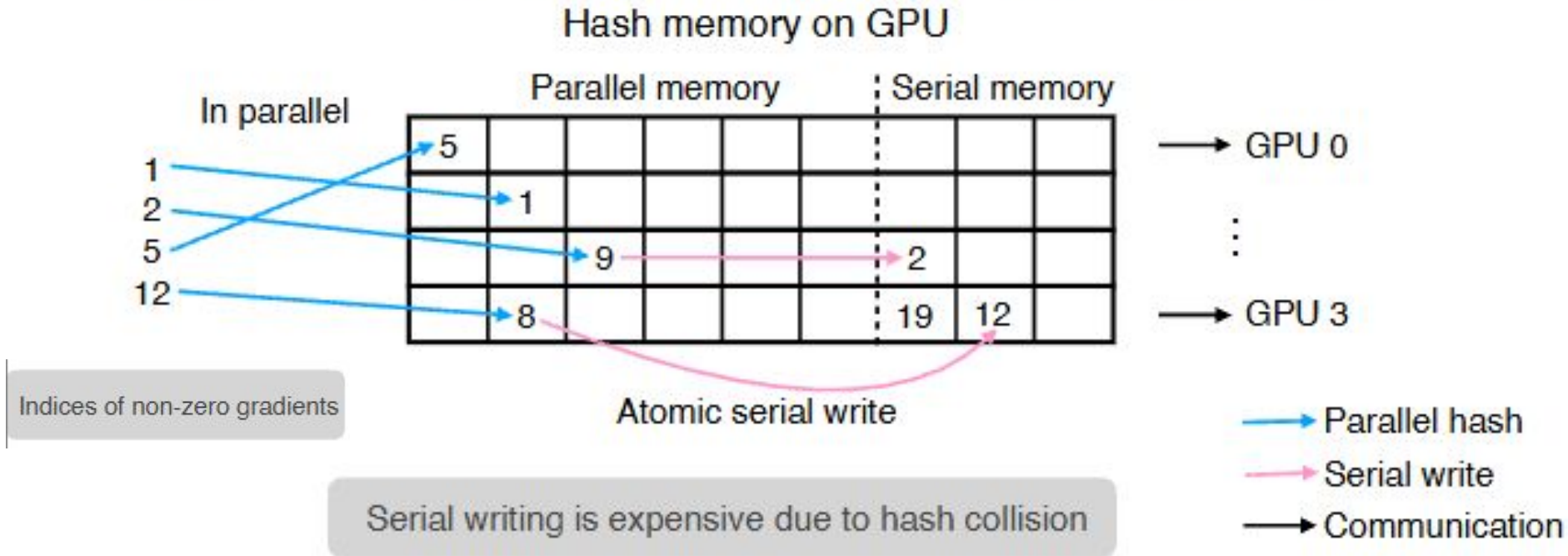
```



A hierarchical hashing algorithm

Balanced Parallelism on GPUs

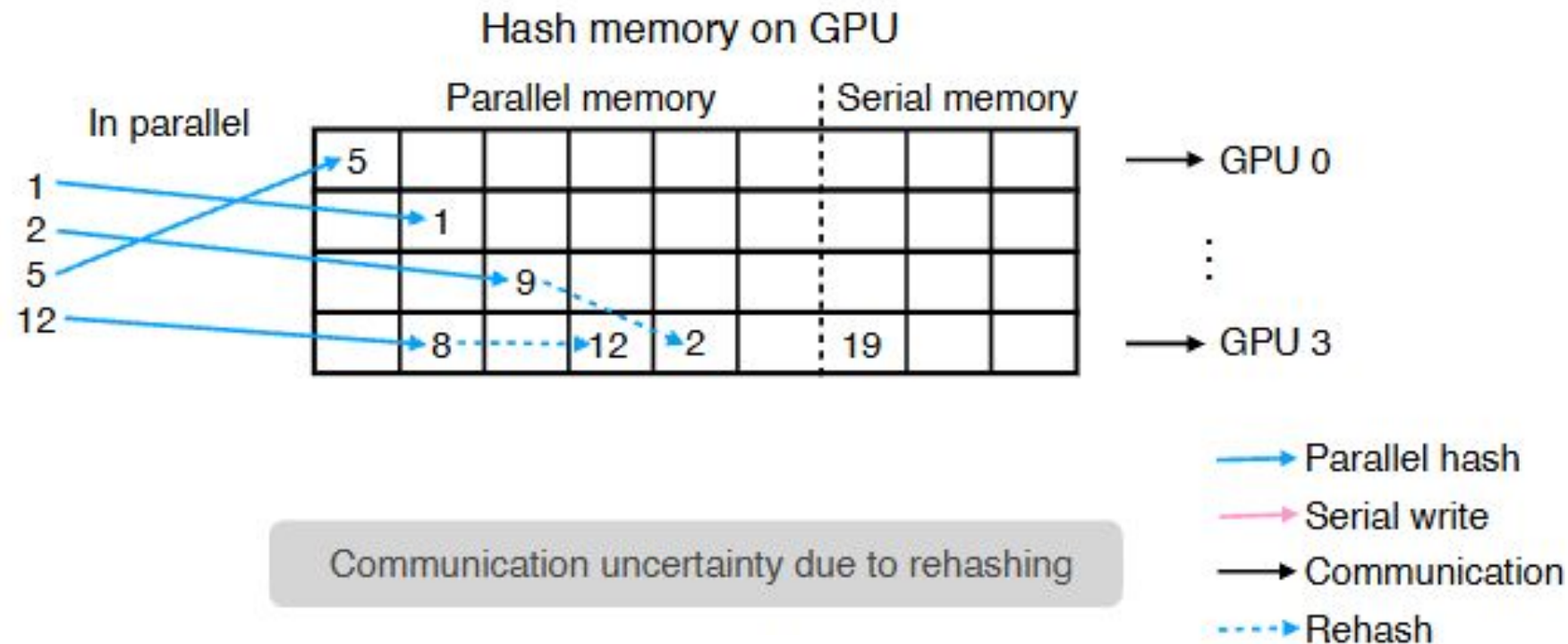
- Technique #1: Communication-oriented hash memory management



A hierarchical hashing algorithm

Balanced Parallelism on GPUs

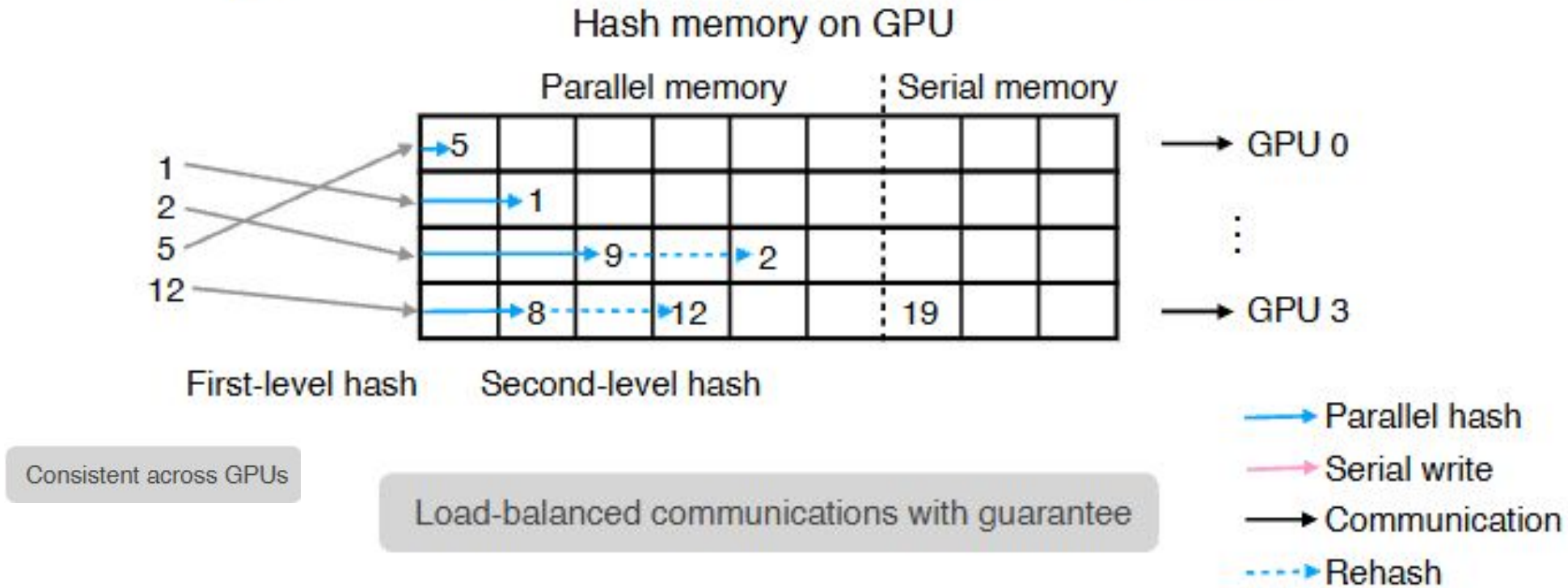
- Technique #2: Multiple hash functions in each GPU thread



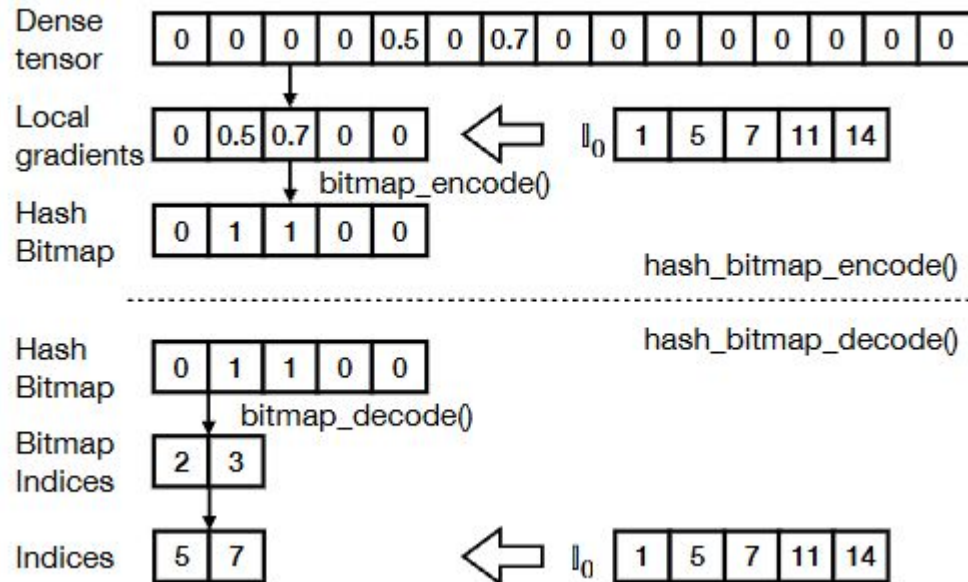
A hierarchical hashing algorithm

Balanced Parallelism on GPUs

- Technique #3: Hierarchical consistent hashing across GPUs



Cost-Efficient Encoding Scheme



Algorithm 2: The hash bitmap

Input: G is a dense tensor. $\mathbb{I}_i = \{idx \in [1, |G|] \mid h_0(idx) = i\}$, where h_0 is defined in Algorithm 1.

Function `hash_bitmap_encode` ($G, \mathbb{I}_i$):

```

    local_gradients =  $G[\mathbb{I}_i]$ 
    hash_bitmap = bitmap_encode(local_gradients)
    return hash_bitmap

```

Function `hash_bitmap_decode` ($\mathbb{I}_i, \text{hash_bitmap}$):

```

    bitmap_indices = bitmap_decode(hash_bitmap)
    indices =  $\mathbb{I}_i[\text{bitmap\_indices}]$ 
    return indices

```



Evaluations

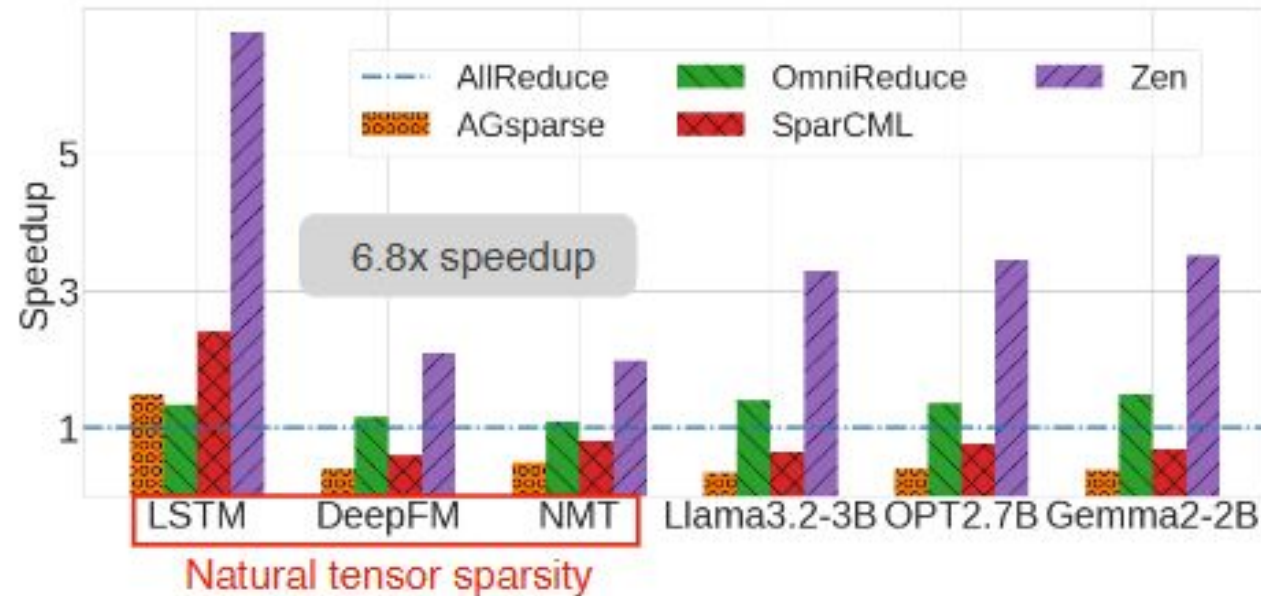
- Two testbeds
 - AWS EC2 p3dn.24xlarge with V100 GPUs, 100 Gbps RDMA networks
 - AWS EC2 p3.16xlarge with V100 GPUs, 25 Gbps TCP/IP networks
- Workloads
 - Models with natural tensor sparsity
 - Models with gradient compression (DGC [1])



Communication improvement

128 V100 GPUs

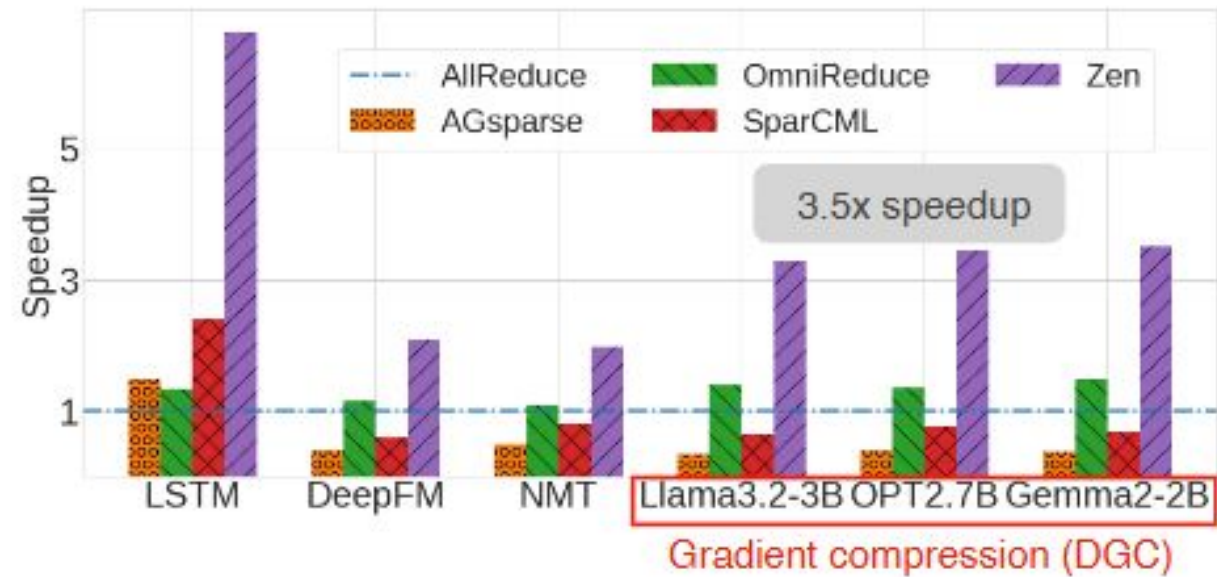
- Communication speedups are normalized to AllReduce



Communication improvement

128 V100 GPUs

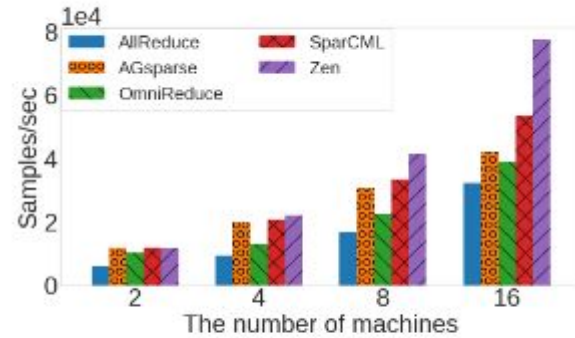
- Communication speedups are normalized to AllReduce



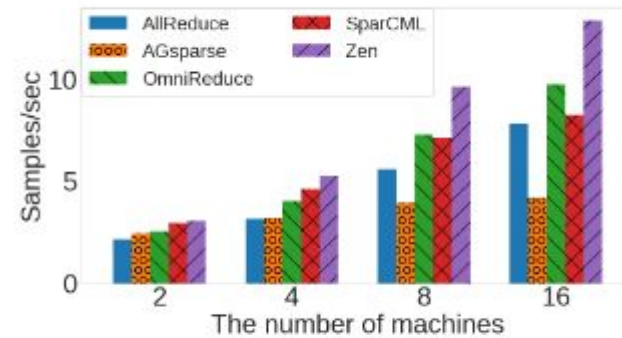
End-to-end training improvement

128 V100 GPUs

- Training throughput



LSTM, natural tensor sparsity
1.5x speedup



Llama3.2-3B, gradient compression
1.3x speedup



Summary

- They study tensor sparsity in popular models.
- They propose Zen with near-optimal synchronization for sparse tensors
 - find provably optimal schemes from the constructed design space
 - propose a hierarchical hashing algorithm for efficient realization on GPUs



Thank You